AGQ: Action-Guided Ultra-Low-Bit Quantization for Vision-Language-Action Models

Xuhong Qiu[1], Zhuoyu Wu[2,*], Qiawei Zheng[3], Wenhui Ou[3], Wenqi Fang[4], Raphael C.W- Phan[2]
[1] Department of Electrical & Computer Engineering, The University of Hong Kong, Hong Kong, China
[2] School of Information Technology, Monash University, Subang Jaya, Malaysia
[3] Department of Electronic & Computer Engineering, Hong Kong University of Science & Technology, Hong Kong, China
[4] Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China
[*]Corresponding author: zhuoyu.wu@monash.edu

***Abstract*—Vision-language-action (VLA) models enable robots to perform tasks specified through natural-language instructions, but their large weight footprint challenges deployment on memory-constrained onboard platforms. Ultra-low-bit quantization reduces this footprint, yet preserving task performance and controlling weight recovery costs remain challenging. We propose Action-Guided Quantization, which combines action sensitivity with a measured proxy for weight recovery cost to allocate matrix bitwidths under storage budgets. Per-channel calibration compensates for quantization error, while a uniform bitwidth within each matrix preserves regular layouts. Bit-dependent grouped lossless coding reduces resident weight storage, and block-wise recovery limits temporary buffers. On LIBERO, AGQ achieves 92.7% task success at W3, exceeding AWQ and GPTQ by 1.2 and 3.7 percentage points, respectively, and retains 70.5% success at W2.5. An asymmetric configuration, denoted W1.5*, uses W4 vision weights and a parameter-weighted average of 1.5 bits for language weights. With lossless coding, it achieves $6.10\times$ complete-model compression relative to BF16 while retaining 25.5% task success.**

***Index Terms*—VLA, mixed-precision quantization, per-channel calibration, lossless coding, hardware-aware inference.**

I. Introduction

Vision-language-action models integrate visual perception, language understanding, and action prediction, enabling robots to perform tasks specified through natural-language instructions [1], [2]. Running these models locally is attractive for robots that operate with limited connectivity or require autonomous execution. However, their large weight footprint poses a challenge for resource-constrained onboard computing platforms. Ultra-low-bit weight quantization offers a way to reduce model storage and make VLA inference feasible under tighter memory budgets. These savings come at the risk of degraded action prediction: in closed-loop control, quantization-induced action errors can affect subsequent observations and task outcomes. Therefore, preserving task performance under aggressive compression remains a central challenge.

Post-training quantization (PTQ) provides a practical approach to compressing pretrained models using a small calibration set [3]. AWQ uses activation statistics to protect salient weights [4], while GPTQ uses second-order information to compensate for quantization error [5]. For VLA models, QVLA and ActQuant further incorporate action-related information into quantization [6], [7]. Beyond calibrating quantized weights, mixed precision allows different layers or matrices to use different bitwidths according to their sensitivity [8], [9]. Hardware-aware approaches such as HAQ additionally incorporate latency and energy feedback into precision selection [10]. Together, these directions provide a basis for allocating precision according to both model behavior and deployment cost.

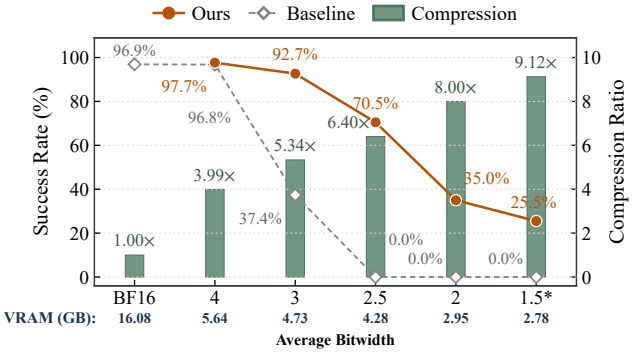


Fig. 1. Bars: target-weight compression. Baseline: QVLA (0% at W2.5; zero extensions at lower bitwidths). 1.5*: W4 vision + W1.5 language.

Ultra-low-bit VLA deployment requires allocating limited precision across matrices with different effects on action prediction. Our analysis reveals sensitivity differences even within the same layer, motivating matrix-wise allocation. The deployment cost also extends beyond average bitwidth: metadata and padding increase storage, while weight recovery requires computation and temporary buffers. Lowering the stored precision alone does not reduce the capacity needed to hold recovered BF16 weights.

To address these issues, we propose **Action-Guided Quantization** (AGQ). AGQ combines action sensitivity with a measured proxy for weight recovery cost to guide matrix-wise bit allocation under storage constraints. Per-channel calibration compensates for quantization error while preserving a uniform bitwidth within each matrix. Grouped lossless coding further reduces resident weight storage, while block-wise recovery limits temporary buffer requirements. Fig. 1 summarizes the resulting tradeoff between task success and target-weight compression.

Our contributions are summarized as follows:

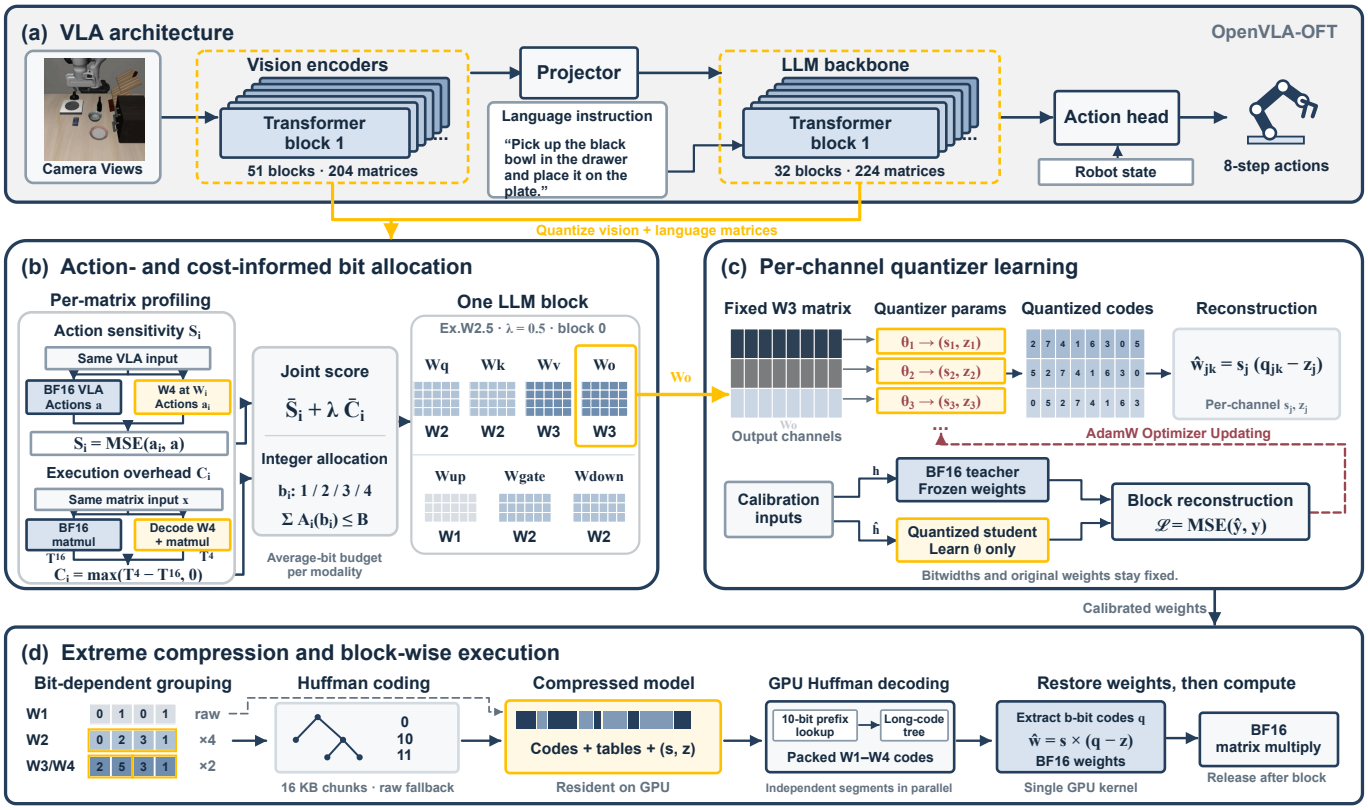


Fig. 2. Overview of AGQ: (a) VLA architecture; (b) action-guided bit allocation incorporating a measured proxy for weight recovery cost; (c) per-channel quantizer calibration; and (d) grouped lossless coding and block-wise execution.

- Action-Guided Quantization, a matrix-wise allocation method that combines an action-sensitivity prior from a common W4 probe with measured weight recovery cost under explicit storage budgets.
- A calibration and execution pipeline that combines per-channel quantizer learning, grouped lossless coding, and block-wise recovery to address quantization error and reduce resident and temporary weight storage.
- Closed-loop evaluation on LIBERO and deployment measurements on RTX3090. AGQ achieves 92.7% and 70.5% overall success at W3 and W2.5, respectively. An asymmetric W4-vision/W1.5-language configuration achieves 25.5% success with 6.10× complete-model compression relative to BF16.

II. Proposed Method

A. Overview

Fig. 2 separates preparation from compressed inference. Starting from a BF16 policy, calibration observations, and backbone storage budgets, stage (b) assigns matrix bitwidths using action sensitivity and execution overhead. Stage (c) fixes this allocation and learns channel quantizers through block reconstruction. Stage (d) encodes the resulting integer weights for deployment. During inference, encoded weights remain GPU-resident; decoding and fused unpacking/dequantization recover the active block's BF16 weights, which are released on exit. Allocation and parameter learning remain offline.

B. Action-Guided Bit Allocation

We assign one bitwidth per matrix to preserve regular decoding. A common W4 perturbation provides a lower-bit allocation prior, assuming some consistency in relative sensitivity across precisions. One probe per matrix replaces separate sensitivity scans at every candidate bitwidth, reducing preparation work without assuming identical rankings.

For matrix i, we quantize only that matrix to W4 and retain all other parameters in BF16. Its sensitivity is the same-input action deviation:

$$S_i = \mathbb{E}_{\text{bal}}\left[\frac{1}{6H}\sum_{h=0}^{H-1}\left\|\mathbf{a}_h^{(i)} - \mathbf{a}_h^{\text{BF16}}\right\|_2^2\right]. \tag{1}$$

Each vector contains six normalized components: three for position and three for rotation, excluding the gripper. With $H = 8$ predicted actions, $6H$ averages over all components and steps. Tasks and their trajectories receive equal weight, preventing longer trajectories from dominating.

Same-input W4 timing provides a measured proxy for weight-recovery overhead. We normalize sensitivity and cost separately within each backbone:

$$C_i = \max\left(T_i^{\text{W4}} - T_i^{\text{BF16}}, 0\right), \tag{2}$$

$$\rho_i = \frac{S_i}{\sum_j S_j} + \lambda \frac{C_i}{\sum_j C_j}. \tag{3}$$

Here λ controls the overhead contribution. Under separate vision and language budgets, we solve

$$\begin{aligned} \min_{\{b_i\}} \quad & \sum_i \rho_i (4 - b_i)^2, \\ \text{s.t.} \quad & \sum_i A_i(b_i) \le B, \quad b_i \in \{1, 2, 3, 4\}, \end{aligned} \tag{4}$$

where $A_i(b_i)$ accounts for candidate-format storage, including metadata and padding, and B is the backbone budget. The quadratic penalty implements a conservative allocation rule: matrices with greater action sensitivity or measured recovery overhead receive stronger protection from aggressive bitwidth reduction. W4 timing sets this priority; it does not predict latency at each candidate precision. Reported compression ratios use actual exported bytes.

C. Per-Channel Calibration for Regular Execution

Per-channel scale s_j sets the quantization step, and integer zero point z_j locates real zero. Sharing them limits metadata and preserves regular access during dequantization.

Following OmniQuant's clipping and block reconstruction [11], we learn clipping coefficients for channel bounds $[\ell_j, u_j]$. For W2–W4, let $Q_i = 2^{b_i} - 1$:

$$s_j = \frac{u_j - \ell_j}{Q_i}, \quad z_j = \mathrm{clip}(\mathrm{round}(-\ell_j / s_j), 0, Q_i)\,. \tag{5}$$

$$q_{jk} = \mathrm{clip}(\mathrm{round}(W_{jk}/s_j) + z_j, 0, Q_i)\,. \tag{6}$$

The codes reconstruct weights as $\widehat{W}_{jk} = s_j(q_{jk} - z_j)$. We floor s_j at a positive constant and round it to BF16 before computing z_j. Weights and bitwidths remain fixed; W1 learns channel amplitudes with signs fixed.

Frozen BF16 block outputs provide reconstruction targets:

$$\mathcal{L}_\ell = \mathbb{E}_{u \in \mathcal{D}}\, \mathrm{MSE}\Big(f_\ell(\widehat{h}_u; \widehat{W}_\ell), f_\ell(h_u; W_\ell) \Big)\,. \tag{7}$$

Here h_u is the teacher input and $\widehat{h}_u$ comes from preceding quantized blocks. AdamW updates the clipping coefficients by backpropagation; the resulting channel bounds determine s_j and z_j, which regenerate quantized weights. Each configuration starts independently from BF16, with a larger calibration share for long-horizon tasks.

Deployment uses these parameters in a GPU kernel that fuses unpacking and per-channel dequantization, followed by BF16 matrix multiplication.

D. Bit-Dependent Grouped Coding and Execution

The allocated bitwidth distribution determines both code statistics and decoding work. We therefore select a grouping granularity for each bitwidth to balance codebook size against the number of values recovered per symbol.

Four adjacent W2 values or two W3/W4 values form a joint symbol for Huffman coding [12]; W1 retains raw packing. These six- or eight-bit symbols capture joint value distributions with at most 256 possible symbols. Each decoded symbol restores multiple values, reducing symbol-processing operations.

For parallel GPU recovery, fixed-size coding blocks are split into independently decodable segments; prefix lookup reduces bit-by-bit traversal. Per-matrix selection compares complete representations, including codebooks and block indexes, with raw packing as fallback. Scales and zero points remain unchanged, and exact code-stream recovery introduces no further weight quantization error.

Encoded weights and decoding metadata remain GPU-resident. On block entry, decoding restores packed codes for fused unpacking/dequantization. Temporary BF16 weights for seven language matrices or four vision matrices are released together on exit. This reduces resident storage and bounds temporary dense weights by the largest active block.

III. Evaluation

A. Experimental Setup

Our setup follows ActQuant [7], using OpenVLA-OFT and four LIBERO suites [13]. Some weights are calibrated on A100; all evaluations run on RTX3090. Ten configurations each run 600 closed-loop episodes (150 per suite), totaling 6,000. We report overall and per-suite success, complete-model storage, peak GPU memory, and mean latency over ten fixed inputs. Each call outputs eight actions; timing excludes observation preprocessing and action postprocessing.

B. Effectiveness of Joint Bit Allocation

Budget	λ	Total (428)			
		W1	W2	W3	W4
W2.5	0	54 (23+31)	107 (67+40)	189 (63+126)	78 (51+27)
	0.25	28 (8+20)	142 (91+51)	192 (55+137)	66 (50+16)
	0.5	19 (6+13)	150 (94+56)	199 (51+148)	60 (53+7)
	0.75	14 (5+9)	155 (96+59)	199 (45+154)	60 (58+2)
	1	7 (5+2)	167 (95+72)	196 (47+149)	58 (57+1)
W2	0	133 (69+64)	111 (42+69)	138 (57+81)	46 (36+10)
	0.25	107 (60+47)	139 (54+85)	149 (59+90)	33 (31+2)
	0.5	95 (55+40)	159 (61+98)	146 (61+85)	28 (27+1)
	0.75	88 (50+38)	164 (69+95)	150 (60+90)	26 (25+1)
	1	86 (49+37)	169 (72+97)	142 (53+89)	31 (30+1)

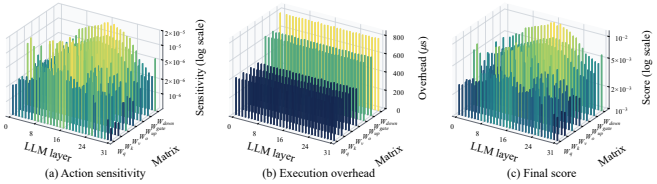


Fig. 3. Allocations (vision + language). Language W4 probes for W2.5, $\lambda = 0.5$: sensitivity and score use logarithmic axes; overhead uses microseconds.

Figure 3 reveals pronounced matrix-level heterogeneity: the plotted action sensitivities span $20.2\times$, while recovery overhead varies $2.0\times$. Differences also occur within a layer, supporting matrix-wise allocation. At W2.5, increasing λ from 0 to 0.5 reduces W1 matrices from 54 to 19; the resulting W1–W4 counts are 19, 150, 199, and 60. Joint scoring thus translates these measured differences into a nonuniform allocation under the storage budget.

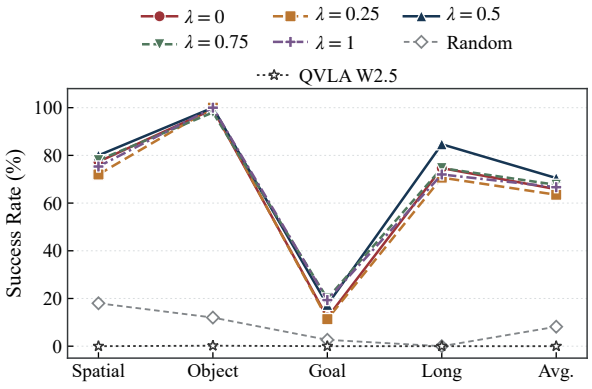


Fig. 4. W2.5 success rates across four suites and overall. Random allocation also undergoes calibration. QVLA values are from ActQuant [7].

The closed-loop results in Fig. 4 confirm the benefit of guided allocation. At W2.5, calibrated random allocation reaches only 8.2% overall success, and the QVLA baseline reaches 0.0%, whereas our five λ settings achieve 63.5–70.5%. This large gap demonstrates the effectiveness of sensitivity-guided assignment within the calibrated pipeline. Among the tested settings, $\lambda = 0.5$ achieves the highest overall success of 70.5%, exceeding sensitivity-only allocation ($\lambda = 0$) by 4.7 percentage points. Its advantage is in the overall average, rather than every suite. Unless otherwise stated, subsequent results use $\lambda = 0.5$.

C. Task Success at Ultra-Low Bitwidths

AWQ and GPTQ: uniform precision; QVLA and Ours: mixed precision (avg. BPW).

Model	Method	Vision+ LLM BPW	Success Rate (%)					
			Spatial	Object	Goal	Long	Avg.	Δ
OpenVLA-OFT	Baseline	16.0	97.6	98.4	96.8	95.1	96.9	0.0
	AWQ	4.0	94.6	98.6	96.8	94.4	96.1	−0.8
		3.0	86.4	98.0	**90.2**	**91.4**	91.5	−5.4
		2.0	0.0	0.0	0.0	0.0	0.0	−96.9
	GPTQ	4.0	90.8	98.8	96.4	92.6	94.6	−2.3
		3.0	82.4	98.0	88.8	87.2	89.0	−7.9
		2.0	0.0	0.0	0.0	0.0	0.0	−96.9
	QVLA	4.0	96.0	98.7	97.4	**95.2**	96.8	−0.1
		3.5	82.9	97.2	91.4	93.6	91.3	−5.6
		3.0	40.8	56.8	20.2	31.8	37.4	−59.5
		2.5	0.0	0.2	0.0	0.0	0.0	−96.9
	Ours	4.0	**97.3**	**100.0**	**98.7**	94.7	**97.7**	**+0.8**
		3.0	**96.7**	**100.0**	83.3	90.7	**92.7**	**−4.2**
		2.5	**80.0**	**100.0**	**17.3**	**84.7**	**70.5**	**−26.4**
		2.0	**29.3**	**92.0**	**1.3**	**17.3**	**35.0**	**−61.9**
		1.5*	38.7	60.7	0.0	2.7	25.5	−71.4

For our most aggressive configuration, the language backbone is reduced to an average of W1.5, while the smaller vision backbone remains at W4 to limit the impact of low-precision visual features on action prediction. Denoted W1.5*, this asymmetric configuration averages approximately 1.75 bits across the targeted vision and language weights.

At shared bitwidths of W3 and below, our method achieves higher overall success than the compared baselines. At W3, its 92.7% success exceeds AWQ, GPTQ, and QVLA by 1.2, 3.7, and 55.3 percentage points, respectively. The advantage becomes more pronounced as the budget shrinks: our method retains 70.5% at W2.5 versus QVLA's 0.0%, and 35.0% at W2 versus 0.0% for both AWQ and GPTQ.

The random-allocation comparison connects this low-bit advantage to informed precision assignment: calibration alone with random bitwidths remains far below the guided configurations. Building on the fixed allocation, calibration is an essential step in our pipeline, using per-channel reconstruction to compensate quantization errors and preserve closed-loop control capability.

Our W1.5* configuration still achieves 25.5% overall success, extending the usable compression range and offering a more aggressive storage–capability tradeoff for memory-constrained compact devices. At W4, overall success reaches 97.7%, slightly above the 96.9% full-precision reference. A possible explanation is a regularizing effect: quantization constrains redundant representations while calibration preserves task-relevant information, potentially mitigating over-representation in the full-precision model.

D. Storage and Inference Efficiency

Model size (compression ratio), peak GPU memory, and latency.

Model	Method	Vision+ LLM BPW	Model Size (GB)	Peak GPU Mem. (GB)	Latency (ms)
OpenVLA-OFT	BF16	16.0	15.457 (1.00×)	16.082	187.3
	Ours	4.0	4.680 (3.30×)	5.643	249.3
		3.0	3.774 (4.10×)	4.727	254.4
		2.5	3.327 (4.65×)	4.280	245.2
		2.0	2.878 (5.37×)	3.828	243.6
		1.5*	2.656 (5.82×)	3.603	250.5
	Ours + Lossless Coding	2.5	3.123 (4.95×)	4.136	473.9
		1.5*	2.534 (6.10×)	3.537	310.4

Without lossless coding, the calibrated W4–W1.5* models execute with fused dequantization in 243.6–254.4 ms (249 ms on average). Thus, lower storage bitwidth does not cause a systematic latency increase across these configurations.

At W1.5*, quantization reduces complete-model storage from 15.457 to 2.656 GB; grouped coding further reduces it to 2.534 GB, giving 6.10× combined compression. Coding adds 59.9 ms over the uncoded configuration, yielding 310.4 ms latency. Peak GPU memory is 3.537 GB, 78.0% below the BF16 reference.

The coded W2.5 configuration takes 473.9 ms, confirming that decoding cost is configuration-dependent. W1.5* combines the smallest complete-model size with the lower latency of the two coded configurations.

For OpenVLA-OFT on RTX4090, published full-inference times are 152.6 ms with NF4, 181.1 ms with AWQ, 181.8 ms with SmoothQuant, and 379.5 ms with LLM.int8() [14]. These LLM-only measurements use rollout timing and provide deployment context, not a matched speedup. Our RTX3090 implementation retains latency in this range with 6.10× complete-model compression, combining practical inference speed with extreme storage reduction.

IV. Conclusion

We presented Action-Guided Quantization (AGQ) for ultra-low-bit VLA deployment. AGQ uses action sensitivity and a measured proxy for weight recovery cost to guide matrix-wise bit allocation, complemented by per-channel calibration, grouped lossless coding, and block-wise recovery. On LIBERO, AGQ outperforms the compared baselines at shared precisions below four bits and achieves 70.5% task success at W2.5. The W1.5* configuration achieves $6.10\times$ complete-model compression while retaining 25.5% success. These results demonstrate a tradeoff between task performance and memory requirements, extending the range of compression options for VLA deployment.

REFERENCES

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09246

[2] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: https://arxiv.org/abs/2502.19645

[3] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021. [Online]. Available: https://arxiv.org/abs/2106.08295

[4] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.17323

[6] Y. Xu, Y. Yang, Z. Fan, Y. Liu, Y. Li, B. Li, and Z. Zhang, "QVLA: Not all channels are equal in vision-language-action model's quantization," in *Int. Conf. Learn. Represent. (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2602.03782

[7] A. Akbari, A. Akbari, M. Eskandar, Q. Tan, Y. Chen, J. Luo, B. Pangaribuan, L. Zhang, J. Dy, G. Yuan, X. Lin, G. Liu, S. Ioannidis, and Y. Wang, "ActQuant: Sub-4-bit action-guided quantization for vision-language-action models," *arXiv preprint arXiv:2605.24011v3*, 2026, version 3, Aug. 3, 2026. [Online]. Available: https://arxiv.org/abs/2605.24011v3

[8] J. Kim, M. Seo, and X. T. Nguyen, "Mixed INT4-INT8 LLM quantization via progressive layerwise assignment with dynamic sensitivity estimation," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11043378/

[9] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. Mahoney, and K. Keutzer, "HAWQ-V3: Dyadic neural network quantization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 11 875–11 886. [Online]. Available: https://proceedings.mlr.press/v139/yao21a.html

[10] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019. [Online]. Available: https://arxiv.org/abs/1811.08886

[11] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *Int. Conf. Learn. Represent. (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2308.13137

[12] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[13] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2306.03310

[14] J. Xu, Q. Jin, M. Chen, S. Wang, Y. Sui, and Y. Shi, "VLAQuantBench: Closed-loop evaluation of post-training quantization for vision-language-action models," *arXiv preprint arXiv:2609.25376*, 2026. [Online]. Available: https://arxiv.org/abs/2609.25376